

Uni-Agent Introduction

Management briefing — design, trajectories, and two demonstrated workflows

Uni-Agent integrates agent harnesses, model inference, sandboxes, and trajectory collection for agentic reinforcement learning. We tested fixed **Qwen3-Coder-30B-A3B-Instruct** weights on AMD ROCm in Docker: **rollouts and evaluation only, with no SFT or RL model update**.

Design and highlights

Component	Responsibility
Task / driver	Select the agent, create and clean up its sandbox, and evaluate the outcome.
Agent / harness	ReAct, real Claude Code CLI, or mini-swe-agent chooses actions using model responses and tool observations.
Gateway	Accept compatible model APIs and record token-level interaction trajectories.
Sandbox provider	Execute commands and file operations in a local Docker or remote environment.
Model backend	vLLM serves the local Qwen30B model on AMD GPUs.

The agentic loop is **observe** → **request the model** → **select a tool** → **execute** → **observe again**, repeated until submission or a stopping limit.

Highlights: **reuse existing harnesses**, **separate inference from tool execution**, and **collect trajectories for training integration**. Claude Code can use local Qwen through Gateway; our E2B integration uses a custom sandbox adapter.

What is a trajectory?

A trajectory records **prompt** → **model response/tool call** → **tool observation** → **next action**. Gateway stores token IDs, generation masks, and optional log probabilities. Masks distinguish model-generated tokens from tool/context tokens for training. Our verifier scores are stored separately and linked by task/session ID.

All **84 main SWE sessions** passed token/mask checks. Their logprobs were not saved; separate runs verified capture for all three harnesses. Collection itself does not train the model.

Case 1 — Docker: repair a real Flask issue

Task: reject empty Blueprint names in SWE-bench issue `pallets__flask-5014`.

The driver created a local task container. ReAct inspected the Blueprint implementation, added validation, ran reproduction checks, and submitted a patch. Requests followed **Uni-Agent Gateway** → **local vLLM**, recording the interaction. A fresh verifier container independently tested the patch.

Result: the issue was resolved: **1 target test and 59 regression tests passed**, with no existing tests modified. [Result and patch](#)

Case 2 — E2B: repair code in a remote sandbox

Task: fix `merge_intervals` for overlapping/nested intervals, input preservation, and invalid ranges.

The driver created an E2B instance through our adapter. Local ReAct chose edits and shell checks; E2B executed them remotely and returned observations. After **13 tool calls**, independent tests were restored and run, then the instance was destroyed.

Result: tests improved from **1/10 to 10/10**. The model remained local. This demo called vLLM directly, without Gateway; it was not remote SWE-bench. [Recorded result](#)

Next milestone: perform a real training update and measure gains on held-out tasks.